

DMBS Lab 10 Demo

Partial Index

A Partial Index is an index built on a subset of the data of the indexed columns. The subset of data is created through where predicates.

- This can be useful when you would like to utilise only a subset of data frequently.
- This can enhance query performance while reducing the index size. It can also improve table updates because PostgreSQL does not need to maintain the index in all cases.
- Keep in mind that partial indices are temporary indices, so as the data access patterns change, change these indices too.

Syntax:

```
create index index_name on table_name(attribute_name) where predicate);
```

Example

```
create index index_store1 on inventory using hash(store_id) where store_id = 1;
explain analyse select title from film
join inventory on film.film_id = inventory.film_id
where inventory.store_id = 1;
```

Output:

Without Index

	QUERY PLAN	
	text	
1	Hash Join (cost=76.50..164.74 rows=2270 width=15) (actual time=1.032..3.872 rows=2270 loops=1)	
2	Hash Cond: (inventory.film_id = film.film_id)	
3	-> Seq Scan on inventory (cost=0.00..82.26 rows=2270 width=2) (actual time=0.022..1.549 rows=2270 loop...	
4	Filter: (store_id = 1)	
5	Rows Removed by Filter: 2311	
6	-> Hash (cost=64.00..64.00 rows=1000 width=19) (actual time=0.996..0.997 rows=1001 loops=1)	
7	Buckets: 1024 Batches: 1 Memory Usage: 60kB	
8	-> Seq Scan on film (cost=0.00..64.00 rows=1000 width=19) (actual time=0.007..0.475 rows=1001 loops=1)	
9	Planning Time: 0.528 ms	
10	Execution Time: 4.133 ms	

With index

	QUERY PLAN
	text
1	Hash Join (cost=76.50..164.74 rows=2270 width=15) (actual time=0.677..2.205 rows=2270 loops=1)
2	Hash Cond: (inventory.film_id = film.film_id)
3	-> Seq Scan on inventory (cost=0.00..82.26 rows=2270 width=2) (actual time=0.015..0.833 rows=2270 loop...
4	Filter: (store_id = 1)
5	Rows Removed by Filter: 2311
6	-> Hash (cost=64.00..64.00 rows=1000 width=19) (actual time=0.653..0.653 rows=1001 loops=1)
7	Buckets: 1024 Batches: 1 Memory Usage: 60kB
8	-> Seq Scan on film (cost=0.00..64.00 rows=1000 width=19) (actual time=0.004..0.296 rows=1001 loops=...
9	Planning Time: 0.438 ms
10	Execution Time: 2.371 ms

Multi-column Index

Index defined on multiple columns

It is also called a composite index, a combined index, or a concatenated index

Only B-tree, GIST, GIN, and BRIN index types support multicolumn indexes.

Syntax:

```
CREATE INDEX index_name ON table_name(attribute1, attribute2, ...);
```

- A multicolumn B-tree index can be used with query conditions that involve any subset of the index's columns, but the index is most efficient when there are constraints on the leading (leftmost) columns. The exact rule is that equality constraints on leading columns, plus any inequality constraints on the first column that do not have an equality constraint, will be used to limit the portion of the index that is scanned.
- It's mandatory to supply the leading column of the index in the query.

Can the non-indexed columns be used with the subset of the indexed columns of an Index?

- It's okay if we skip an index column, but remember the leading **column of the index is always required**. If an index is defined as Index(a, b, c) and a query skips on a search on a,b, then the query plan looks up the values of a, then c, ignoring the values of b
- However, skipping columns during index scans (index skip scans) is expensive.

Does the order of the columns defined in the index matter?

- Order of the columns placed in the index matters: place them in descending order of access frequency. The query optimizer can use the index when the query's conditions involve the index's leading (leftmost) column.
- If a query does *not* constrain the first column of the index, PostgreSQL must evaluate whether a full index scan on this index is more efficient than alternative indexes or a table scan. The planner evaluates different execution paths, and if an index scan is not the most efficient, it may choose a sequential scan instead.
- **The order of columns in the where clause matters for speed**
- It simply says to use the columns with unique values at the start of the index because duplicate values increase the search space, even if we supply the exact match in the WHERE clause.

How to efficiently use indices:

1. Write all the possible queries that you are going to run optimally, and list down all the columns used.
2. List the columns in the where predicate in the order:
 - a. Need an exact match
 - b. Needed in-range query
 - c. Have duplicate values

Example

: Query:

```
create index mul_index on rental(customer_id, return_date);
explain analyse select title from film
join inventory on film.film_id = inventory.film_id
join rental on inventory.inventory_id = rental.inventory_id
where customer_id = 87 and return_date between '2005-06-29' and
'2005-08-25';
```

Output:

Without index

	QUERY PLAN
	text
1	Nested Loop (cost=0.84..527.78 rows=16 width=15) (actual time=0.132..0.699 rows=18 loops=1)
2	-> Nested Loop (cost=0.57..522.27 rows=16 width=2) (actual time=0.123..0.624 rows=18 loops=1)
3	-> Index Scan using idx_unq_rental_rental_date_inventory_id_customer_id on rental (cost=0.29..421.47 rows=16 width=4) (actual time=0.113..0.541 rows=18 loops=1)
4	Index Cond: (customer_id = 87)
5	Filter: ((return_date >= '2005-06-29 00:00:00'::timestamp without time zone) AND (return_date <= '2005-08-25 00:00:00'::timestamp without time zone))
6	Rows Removed by Filter: 12
7	-> Index Scan using inventory_pkey on inventory (cost=0.28..6.30 rows=1 width=6) (actual time=0.004..0.004 rows=1 loops=18)
8	Index Cond: (inventory_id = rental.inventory_id)
9	-> Index Scan using film_pkey on film (cost=0.28..0.34 rows=1 width=19) (actual time=0.003..0.003 rows=1 loops=18)
10	Index Cond: (film_id = inventory.film_id)
11	Planning Time: 0.666 ms
12	Execution Time: 0.742 ms

With index

	QUERY PLAN
	text
1	Nested Loop (cost=5.05..159.40 rows=16 width=15) (actual time=0.055..0.186 rows=18 loops=1)
2	-> Nested Loop (cost=4.77..153.89 rows=16 width=2) (actual time=0.049..0.134 rows=18 loops=1)
3	-> Bitmap Heap Scan on rental (cost=4.49..53.09 rows=16 width=4) (actual time=0.038..0.064 rows=18 loops=1)
4	Recheck Cond: ((customer_id = 87) AND (return_date >= '2005-06-29 00:00:00'::timestamp without time zone) AND (return_date <= '2005-08-25 00:00:00'::timestamp without time zone))
5	Heap Blocks: exact=16
6	-> Bitmap Index Scan on mul_index (cost=0.00..4.49 rows=16 width=0) (actual time=0.031..0.032 rows=18 loops=1)
7	Index Cond: ((customer_id = 87) AND (return_date >= '2005-06-29 00:00:00'::timestamp without time zone) AND (return_date <= '2005-08-25 00:00:00'::timestamp without time zone))
8	-> Index Scan using inventory_pkey on inventory (cost=0.28..6.30 rows=1 width=6) (actual time=0.003..0.003 rows=1 loops=18)
9	Index Cond: (inventory_id = rental.inventory_id)
10	-> Index Scan using film_pkey on film (cost=0.28..0.34 rows=1 width=19) (actual time=0.002..0.002 rows=1 loops=18)
11	Index Cond: (film_id = inventory.film_id)
12	Planning Time: 0.704 ms
13	Execution Time: 0.229 ms

While multicolumn indexes can improve query performance, they also add overhead to write operations (**INSERT**, **UPDATE**, **DELETE**).

Covering Index

A covering index allows a user to perform an index-only scan if the select list in the query matches the columns that are included in the index.

You can specify the additional columns for the index using the "INCLUDE" keyword.

If you have a query:

```
SELECT name, age FROM users WHERE city = 'New York';
```

A regular index on **city** helps find the row, but the engine must look up **name** and **age** in the main table.

A **covering index** created as **CREATE INDEX idx_city_covering ON users (city) INCLUDE (name, age);** allows the engine to get **name** and **age** directly

from the index.

Syntax i:

```
CREATE INDEX index_name ON table_name(attribute1, attribute2, ...) INCLUDE  
(attribute_k);
```

Example

```
CREATE INDEX name_email on table_name(customer_id) include (payment_date);  
SELECT * from payment where customer_id = 87 and payment_date between  
"2007-03-20" and "2007-04-30";
```

Output:

With index

	QUERY PLAN text	🔒
1	Bitmap Heap Scan on payment (cost=4.46..61.08 rows=12 width=6) (actual time=0.028..0.033 rows=13 loops=1)	
2	Recheck Cond: (customer_id = 87)	
3	Filter: ((payment_date >= '2007-03-20 00:00:00'::timestamp without time zone) AND (payment_date <= '2007-04-30 00:00:00'::timestamp without time z...	
4	Rows Removed by Filter: 15	
5	Heap Blocks: exact=4	
6	-> Bitmap Index Scan on idx_fk_customer_id (cost=0.00..4.46 rows=23 width=0) (actual time=0.016..0.016 rows=28 loops=1)	
7	Index Cond: (customer_id = 87)	
8	Planning Time: 0.115 ms	
9	Execution Time: 0.054 ms	

Without index

	QUERY PLAN text	🔒
1	Bitmap Heap Scan on payment (cost=4.46..61.08 rows=12 width=6) (actual time=0.088..0.104 rows=13 loops=1)	
2	Recheck Cond: (customer_id = 87)	
3	Filter: ((payment_date >= '2007-03-20 00:00:00'::timestamp without time zone) AND (payment_date <= '2007-04-30 00:00:00'::timestamp without time z...	
4	Rows Removed by Filter: 15	
5	Heap Blocks: exact=4	
6	-> Bitmap Index Scan on idx_fk_customer_id (cost=0.00..4.46 rows=23 width=0) (actual time=0.050..0.051 rows=28 loops=1)	
7	Index Cond: (customer_id = 87)	
8	Planning Time: 0.180 ms	
9	Execution Time: 0.158 ms	

This can reduce the amount of I/O your query needs to use in order to retrieve information. Additionally, the data types included in a covering index do not need to be B-tree indexable; you can add any data type to the `INCLUDE` part of a `CREATE INDEX` statement.

Each column you add to the index still takes up space on disk, and there is still a cost for maintaining the index, for example, on row updates. Understanding these trade-offs, you can still apply covering indexes in very helpful ways that can significantly help your applications.

In PostgreSQL, a B-Tree index creates a multi-level tree structure where each level can be used as a doubly-linked list of pages. Leaf pages are those at the lowest level of a tree that point to rows of tables. With *covering indexes*, records of the columns mentioned in the `INCLUDE` clause are included in the leaf pages of the B-Tree as "payload" and are not part of the search key.

Advantages of using Indexes with an `INCLUDE` clause:

1. The B-tree has fewer levels because they do not contain include columns
2. Greatly improves performance
3. Has the ability to return the contents of non-key columns without having to visit the index's table

Limitations of using Indexes with included columns

1. Expressions are not supported as included columns since they cannot be used in index-only scans.
2. Only B-tree, GiST and SP-GiST indexes currently support included columns.
3. It's wise to be conservative about adding non-key payload columns to an index, especially wide columns. If an index tuple exceeds the maximum size allowed for the index type, data insertion will fail.
4. Non-key columns duplicate data from the index's table and bloat the size of the index, thus potentially slowing searches

Unique Index

The PostgreSQL unique index enforces the uniqueness of values in one or multiple columns. PostgreSQL offers multiple index types, but **only the B-tree index type supports unique indexes**.

When you define a unique index for a column, the column cannot store multiple rows with the same values. If you define a unique index for two or more columns, the combined values in these columns cannot be duplicated in multiple rows.

When you define a primary key or a unique constraint for a table, PostgreSQL automatically creates a corresponding unique index. **UNIQUE** indexes can be combined with other constraints, such as `NOT NULL`, to enforce stricter data integrity rules. There's no need to manually create indexes on unique columns; doing so would just duplicate the automatically-created index.

Syntax:

```
create unique index index_name on table_name(attribute1);
```

Using UNIQUE indexes in PostgreSQL offers several benefits:

1. **Data Integrity:** Ensures no duplicate values are allowed in the specified columns.
2. **Performance:** Speeds up search queries for unique fields by reducing the need for additional duplicate data checks.
3. **Efficient Queries:** Enables efficient lookups for unique entries like email addresses or user IDs, often used in WHERE or JOIN clauses.

Index on Expressions

PostgreSQL also allows you to create an index based on an expression involving table columns. This type of index is called an **index on expression / functional indexes**.

After defining an index expression, PostgreSQL will consider using that index when the expression appears in the `WHERE` clause or in the `ORDER BY` clause of the SQL statement. Note that maintaining indexes on expressions can incur additional costs. PostgreSQL evaluates the expression for each row during insertion or update and utilizes the result for building the index.

Not every expression needs an index. Focus on queries that are run frequently and benefit from faster execution times. You can create indexes on more complex expressions or even on multiple columns. However, ensure that the expression is used consistently in your queries.

There is another benefit to expression indexes, and that is optimizer statistics. Not only do expression indexes allow rapid lookups of matching expressions, but they also provide optimizer statistics, which improve row estimates and hence query plans.

Syntax:

```
CREATE INDEX index_name ON table_name(expression(attributes));
```

Example

: Query:

```
create index actor_names on actor((first_name || ' ' || last_name));
explain analyse select title from film
join film_actor on film.film_id = film_actor.film_id
join actor on film_actor.actor_id = actor.actor_id
where (first_name || ' ' || last_name) = 'Penelope Guinness';
```

Output: With index:

	QUERY PLAN	
	text	
1	Nested Loop (cost=4.77..50.30 rows=27 width=15) (actual time=0.056..0.190 rows=19 loops=1)	
2	-> Nested Loop (cost=4.49..41.24 rows=27 width=2) (actual time=0.048..0.116 rows=19 loops=1)	
3	-> Seq Scan on actor (cost=0.00..5.50 rows=1 width=4) (actual time=0.015..0.075 rows=1 loops=1)	
4	Filter: (((first_name)::text ' '::text) (last_name)::text) = 'Penelope Guinness'::text)	
5	Rows Removed by Filter: 199	
6	-> Bitmap Heap Scan on film_actor (cost=4.49..35.47 rows=27 width=4) (actual time=0.029..0.033 rows=19 loops=1)	
7	Recheck Cond: (actor_id = actor.actor_id)	
8	Heap Blocks: exact=1	
9	-> Bitmap Index Scan on film_actor_pkey (cost=0.00..4.48 rows=27 width=0) (actual time=0.008..0.008 rows=19 loop...	
10	Index Cond: (actor_id = actor.actor_id)	
11	-> Index Scan using film_pkey on film (cost=0.28..0.34 rows=1 width=19) (actual time=0.003..0.003 rows=1 loops=19)	
12	Index Cond: (film_id = film_actor.film_id)	
13	Planning Time: 1.217 ms	
14	Execution Time: 0.243 ms	

Without index:

	QUERY PLAN	
	text	
1	Nested Loop (cost=4.77..50.30 rows=27 width=15) (actual time=0.031..0.133 rows=19 loops=1)	
2	-> Nested Loop (cost=4.49..41.24 rows=27 width=2) (actual time=0.027..0.076 rows=19 loops=1)	
3	-> Seq Scan on actor (cost=0.00..5.50 rows=1 width=4) (actual time=0.012..0.056 rows=1 loops=1)	
4	Filter: (((first_name)::text ' '::text) (last_name)::text) = 'Penelope Guinness'::text)	
5	Rows Removed by Filter: 199	
6	-> Bitmap Heap Scan on film_actor (cost=4.49..35.47 rows=27 width=4) (actual time=0.012..0.014 rows=19 loops=1)	
7	Recheck Cond: (actor_id = actor.actor_id)	
8	Heap Blocks: exact=1	
9	-> Bitmap Index Scan on film_actor_pkey (cost=0.00..4.48 rows=27 width=0) (actual time=0.006..0.006 rows=19 loop...	
10	Index Cond: (actor_id = actor.actor_id)	
11	-> Index Scan using film_pkey on film (cost=0.28..0.34 rows=1 width=19) (actual time=0.002..0.002 rows=1 loops=19)	
12	Index Cond: (film_id = film_actor.film_id)	
13	Planning Time: 0.573 ms	
14	Execution Time: 0.170 ms	

Combining Multiple Indices

A single index scan can only use query clauses that use the index's columns with operators of its operator class and are joined with AND. For example, given an index on (a, b) there are two cases:

1. a query condition like WHERE a = 5 AND b = 6 could use the index
2. A query condition like WHERE a = 5 OR b = 6 could not directly use the index.

But, a query like `WHERE x = 42 OR x = 47 OR x = 53 OR x = 99` could be broken down into four separate index scans, each using one of the query clauses. The results of these scans are then ORed together to produce the result.

Another example is that if we have separate indexes on x and y, one possible implementation of a query like `WHERE x = 5 AND y = 6` is to use each index with the appropriate query clause and then `AND` together the index results to identify the result rows.